

নির্বাহী সারাংশ

এই রিপোর্টে আমরা ফেজ ১ (**MVP**) এর জন্য একটি শিক্ষক-নিয়ন্ত্রিত **AI**-সমর্থিত পাইথন মূল্যায়ন প্ল্যাটফর্মের ডিজাইন ও বাস্তবায়ন পরিকল্পনা বিশ্লেষণ করেছি। প্ল্যাটফর্মটি দুটি প্রধান ব্যবহারকারী গ্রুপের (শিক্ষক এবং শিক্ষার্থী) জন্য থাকবে এবং সম্পূর্ণ ফ্রন্টএন্ড-ভিত্তিক। মূল বৈশিষ্ট্যসমূহের মধ্যে রয়েছে: শিক্ষক ড্যাশবোর্ড, **AI**-জেনারেটেড প্রশ্ন, শিক্ষক দ্বারা প্রশ্ন পর্যালোচনা/সম্পাদনা, কুইজ তৈরি (বিভাগ/সেট নির্বাচন), এবং ছাত্রদের লাইভ কুইজ সেশন (কোড দিয়ে যোগদান)। প্রতিটি প্রশ্নের জন্য ২০ সেকেন্ড টাইমার থাকবে এবং সময় ফুরিয়ে গেলে স্বয়ংক্রিয়ভাবে পরবর্তী প্রশ্নে চলে যাবে। ৩০% এর বেশি ভুলের ক্ষেত্রে নেতিবাচক মার্কিং (-0.25) কার্যকর হবে। কুইজ শেষে বিস্তারিত ফলাফল ও বিশ্লেষণ (দক্ষতা-বিভাগে ফলাফল) দেখানো হবে। পাশাপাশি, প্রাথমিক ধাপে কিছু মৌলিক অ্যাক্টিভিটি ব্যবস্থা (ট্যাব-সুইচ সনাক্তকরণ, ফুলক্রিন বাধ্যতামূলক) থাকবে। সবকিছু ফ্রন্টএন্ডে স্ট্যাটিকভাবে (JSON/TS ফাইল) রাখা হবে, তবে ভবিষ্যতে ব্যাকএন্ড/ডাটাবেস সহজে সংযুক্ত করতে সক্ষম ডিজাইনে রাখা হবে।

মূল উপাত্তগুলো নিম্নরূপ সাজানো হয়েছে: (1) ইমপ্লিমেন্টেশন প্রস্পেক্ট (ডেভ/আই এজেন্টের জন্য), (2) **UI** ও উপাদান তালিকা, (3) ডেটা স্কিমা ও উদাহরণ **JSON**, (4) প্রশ্ন **QA** চেকলিস্ট, (5) **MVP** রোডম্যাপ ও সময়রেখা, (6) নিরাপত্তা/সততা বিবেচনা (**Phase ১** ও **Phase ২**), এবং (7) মূল্যায়ন সূচক। তদুপরি, শিক্ষক ফ্লো, ছাত্র ফ্লো, ডেটা ফ্লো এর জন্য **Mermaid** ডায়াগ্রাম যুক্ত করা হয়েছে। প্রতিটি ধাপে গবেষণাগত উৎস থেকে প্রাপ্ত তথ্য ব্যবহার করা হয়েছে এবং সংযত উদ্ধৃতিসহ বিস্তারিত বর্ণনা করা হয়েছে।

ইমপ্লিমেন্টেশন প্রস্পেক্ট (**AI** এজেন্টের জন্য)

```
Build an end-to-end frontend-only Python quiz assessment platform (React/
Next.js + TypeScript + Tailwind).
This MVP is for Phase 1 targeting course teachers (admins) and students,
based on the given syllabus sections.
Key features:
- Teacher Dashboard with stats (students, quizzes, questions).
[DashboardStats, SectionList]
- Create Quiz flow: teacher selects section & set, number of questions (30
total: 10 easy, 10 medium, 10 hard), time limit (20s/q), negative marking
(>30% wrong triggers -0.25 per wrong).
- AI-powered question generation: When teacher creates a quiz, generate
appropriate MCQs for chosen topics and difficulties. Each question must
include question text, optional code snippet, 4 options with exactly one
correct answer, difficulty label, topic label, and explanation. Use Python 3
logic to verify correctness.
- Human-in-the-loop review: Show AI-generated questions in a review list.
Provide actions like "Edit", "Regenerate", "Make Harder", "Make Easier",
"Regenerate Distractors" etc. Teacher must approve/edit all questions before
publishing. Enforce one correct answer and clear phrasing [1†L122-L124] .
- Publish Quiz: After review, quiz is published and a join-code is
generated.
- Student flow: Students join by entering the quiz code and their name. They
wait in a "waiting room" until the teacher starts the quiz.
- Quiz Interface: Display one question at a time, with a visible **20-second
countdown timer** (per question). The UI shows "Question X / 30" and a
progress indicator. Timer should count down prominently (e.g. progress bar or
numbers) [3†L50-L53] . When timer hits 0, mark question unanswered and auto-
move to next. Do not allow answering after time expires.
```

- Answer behavior: When a student clicks an option, record the answer and immediately advance to the next question (or show a Next button). Do NOT reveal correctness during the quiz.
- Scoring: 1 point per correct answer. Wrong answers have no penalty *unless* wrong count > 30% of total. If wrong answers > 9 (for 30 Q), then apply -0.25 for each wrong. Show scoring rules on instruction screen.
- Results Screen: After quiz, display final score and detailed stats: total Q (30), correct, wrong, unanswered, accuracy %. Also break down performance by difficulty (easy/med/hard) and by topic. Provide a question-by-question review page: for each Q show question, code (if any), options, user's answer, correct answer, correctness indicator, explanation, topic, difficulty.
- Data & Questions: Organize questions by files (e.g. `questions/sectionX/setY.json` or `.ts`). Each question object: `{ id, question, code, options[4], correctAnswer(index), difficulty, topic, explanation }`. Use static JSON/TS for now (frontend), but structure it so a backend could supply this in future.
- UI Pages & Components: Design pages for Teacher Dashboard, Quiz Creation & Review, Quiz Instructions, Student Join, Waiting Room, Quiz Question, Results, Review. Create reusable components (e.g. SectionCard, QuizSetCard, QuizQuestion, OptionButton, TimerBar, ProgressBar, ResultSummary).
- State management: Use React state/Context or a lightweight state manager to track current quiz, questions, answers, timer, score. Ensure state persists correctly across component re-renders. Handle router transitions cleanly.
- Tech stack: React + Next.js (for SPA with static export), TypeScript, Tailwind CSS for styling. No backend required for Phase 1; load question data from local static files.
- Timer & Concurrency: Implement timer logic so that setInterval accurately counts down 20s/q. (Note: avoid timer drift on tab switch by using Web Worker or performance.now [12†L49-L57] [12†L61-L64] .) Reset timer on each new question.
- Anti-cheat: Use the Page Visibility API to detect if the user switches tabs (`visibilitychange` event) or loses focus [19†L184-L193] . Log or mark any such events (don't auto-fail, but record them). Prompt full-screen mode on start to minimize switching. Disable copy-paste of code.
- Accessibility & UX: Ensure keyboard navigation and screen-reader support (use semantic HTML ``, `aria-live` for timer updates, etc.). Use high-contrast colors and visible focus states [6†L309-L317] . UI should be responsive.
- Testing & Edge Cases: Disable multiple rapid clicks (debounce answers). If answer is selected at the exact moment timer hits 0, ensure no double-scoring. Handle refresh (ideally warn user it will restart or abort quiz). Validate question data integrity (no missing fields). Confirm negative-marking threshold logic (only apply penalty if wrong > 9).
- Deliverable: Provide a clean, modular frontend codebase. Include the full question bank (12 sets×30 Qs, categorized). Provide the above features end-to-end in a demo-ready UI.

UI/UX ও উপাদানসমূহ

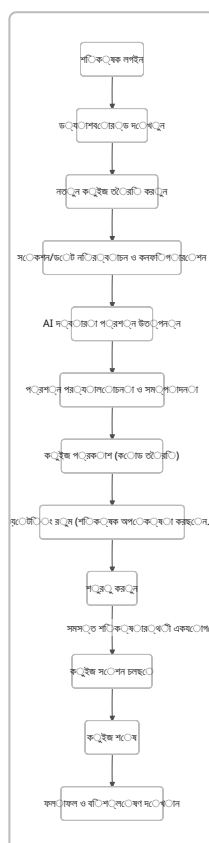
স্ক্রিন (ওয়ারফ্রেম) তালিকা:

- **Teacher Dashboard:** সারসংক্ষেপ (মোট ছাত্র, কুইজ, প্রশ্ন সংখ্যা) দেখাবে; “+Quiz তৈরি করুন” বাটন।
- **Quiz Creation / AI** জেনারেট: শিক্ষক বিভাগ ও সেকশন নির্বাচন করে কুইজ কনফিগার করে (প্রশ্ন সংখ্যা, টাইমার, নিউ মার্কিং ইত্যাদি)। “AI Generate Questions” বাটন।
- **Question Review** ও সম্পাদনা: AI-জেনারেট প্রশ্নের তালিকা, প্রতিটি প্রশ্নে “Edit” বা “Regenerate” অপশন। টেবিল বা কার্ডের মাধ্যমে প্রশ্ন, অপশন, এবং সম্পাদনার UI।
- **Quiz Instructions:** শুরু করার আগে নোটিশ (30 প্রশ্ন, 20s পি. কিউ., মার্কিং স্কীম)।
- **Student Join Page:** কুইজ কোড এবং নাম দিয়ে জয়েন।
- **Waiting Room:** যোগ দেয়া ছাত্রদের নাম দেখাবে, শিক্ষক “Start Quiz” অর্দি অপেক্ষা।
- **Quiz Screen:** একক প্রশ্ন প্রদর্শন – কোড ব্লক (যদি থাকে), 4টি বিকল্প বোতাম, ২০ সেকেন্ড কাউন্টডাউন (সংখ্যা ও/অথবা বার গ্রাফ)। উপরে প্রশ্ন অগ্রগতি (উদাঃ “প্রশ্ন 5/30”)।
- **Result Screen:** মোট স্কোর, সঠিক/ভুল/নিশ্চয় কত, শতকরা সঠিকতা দেখাবে। কঠিন / মধ্যম / সহজ প্রতি বিভাগে ফলাফল দেখাবে।
- **Review Screen:** প্রতিটি প্রশ্নের জন্য ফিডব্যাক: প্রশ্ন, কোড (যদি থাকে), সমস্ত অপশন, শিক্ষার্থীর উত্তর, সঠিক উত্তর, ফলাফল (✓/✗), এবং বিস্তারিত ব্যাখ্যা।

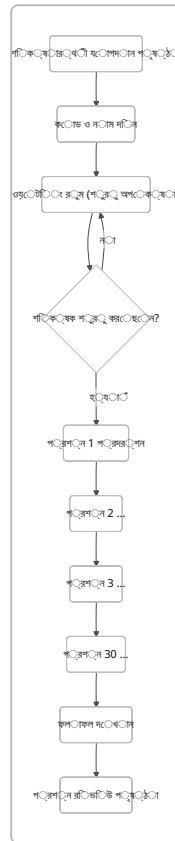
UI উপাদান (Components):

উপাদান (Component)	বিবরণ
DashboardStats	সারাংশ কার্ড: মোট ছাত্র, মোট কুইজ, মোট প্রশ্ন ইত্যাদি দেখায়।
SectionCard	প্রতিটি সেকশনের সারসংক্ষেপ (নাম, সেট সংখ্যা, মোট প্রশ্ন)। “[Explore Sets]” বাটন।
QuizSetCard	সেকশনের প্রতিটি সেটের জন্য। (উদাঃ “Set 1 – 30 Questions, 10E/10M/10H”).
QuizInstructions	কুইজ শুরুর আগে নির্দেশাবলী। (+1/-0.25 মার্কিং টেবিল ইত্যাদি)
QuizQuestion	প্রশ্ন এবং কোড ব্লক (যদি থাকে) রেন্ডার করে। পর্যাপ্ত মার্জিন ও মনোস্পেস।
OptionButton	একক উত্তর অপশন বোতাম; সিলেক্টেড/নিষ্ক্রিয়/ভুল স্টেট প্রদর্শন।
QuizTimer	২০ সেকেন্ড কাউন্টডাউন (সংখ্যা ও/অথবা প্রশ্নের বার) দেখায়।
ProgressBar	প্রশ্ন অগ্রগতি দেখায় (“Q7/30”) – বার গ্রাফ বা সংখ্যাসূচক ইঙ্গিত।
ResultSummary	চূড়ান্ত ফলাফলের সারাংশ (সঠিক, ভুল, নিশ্চয়, অ্যাকুরেসি) UI।
QuestionReviewItem	রিভিউ পেজের একটি প্রশ্নের আইটেম (প্রশ্ন + অপশন + সঠিক উত্তর + ব্যাখ্যা)।

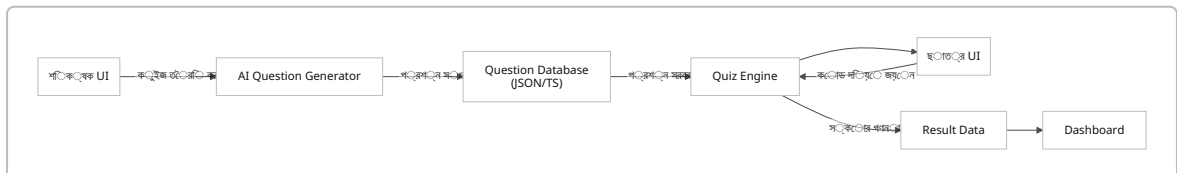
শিক্ষকের কার্যপ্রবাহ (Teacher Flow)



ছাত্রের কার্যপ্রবাহ (Student Flow)



তথ্য প্রবাহ (Data Flow)



প্রশ্ন ডেটা কাঠামো ও উদাহরণ

প্রতিটি প্রশ্ন স্ট্যাটিক ফাইল/ওবজেক্টে রাখা হবে। ফাইল স্ট্রাকচার:

```

questions/
├── section1/
│   ├── set1.ts
│   ├── set2.ts
│   └── set3.ts
├── section2/
│   ├── set1.ts
│   ├── set2.ts
│   └── set3.ts
  
```

```
└─ section3/ ...
└─ section4/ ...
```

প্রতিটি ফাইলে ৩০টি প্রশ্ন (10 easy, 10 medium, 10 hard) থাকবে। প্রশ্ন অবজেক্টের কাঠামো:

```
{
  id: string;
  question: string;
  code?: string;
  options: string[4];
  correctAnswer: number;
  difficulty: "easy"|"medium"|"hard";
  topic: string;
  explanation: string;
}
```

উদাহরণ (section1/set1.json থেকে 3টি প্রশ্ন):

```
[
  {
    "id": "sec1-set1-q01",
    "question": "নীচের কোডটি চালালে কী আউটপুট হবে?",
    "code": "a = [1, 2, 3]\nb = a\na.append(4)\nprint(b)",
    "options": [
      "[1, 2, 3]",
      "[1, 2, 3, 4]",
      "[4]",
      "Error"
    ],
    "correctAnswer": 1,
    "difficulty": "easy",
    "topic": "Object & Reference",
    "explanation":
      "b এবং a একই লিস্টের রেফারেন্স। তাই b.append(4) করলে a এর ভিতরও পরিবর্তন হয়। তাই b এর মূল্য হবে [1,2,3,4]।"
  },
  {
    "id": "sec1-set1-q02",
    "question": "নীচের কোডটি কি প্রিন্ট করবে?",
    "code": "text = 'Python'\nprint(text[::-1])",
    "options": [
      "nohtyP",
      "Python",
      "Error",
      "TnhoP"
    ],
    "correctAnswer": 0,
    "difficulty": "medium",
    "topic": "Slicing",
    "explanation": "text[::-1] স্ট্রিংটি উল্টো করে। তাই 'Python' উল্টো হলে 'nohtyP' পাওয়া যায়।"
```

```

},
{
  "id": "sec1-set1-q03",
  "question": "নীচের কোডটির আউটপুট কী হবে?",
  "code": "def func(x=[]):\n    x.append(1)\n    return x\nprint(func(),\nfunc())",
  "options": [
    "[1] [1]",
    "[1, 1] [1, 1]",
    "[1] [1, 1]",
    "[1, 1] [1]"
  ],
  "correctAnswer": 2,
  "difficulty": "hard",
  "topic": "Function / Default Arguments",
  "explanation": "ডিফল্ট লিস্ট x হল mutable। প্রথমবার func() কল করলে [1] ফেরত দেয়, কিন্তু ঐ লিস্টে 1 যোগ হয়েছে। দ্বিতীয়বার func() ডাকা হলে একই লিস্টে আবার 1 যোগ হবে। তাই আউটপুট হবে \"[1] [1, 1]\"।"
}
]

```

উপরের উদাহরণে কোড-আউটপুটগুলি Python 3 এ যাচাই করা। প্রতিটি প্রশ্নের `correctAnswer` একটি ইনডেক্স (0-3) যা সঠিক অপশনের অবস্থান। `explanation` -এ সঠিক উত্তরের কারণ ও বিষয়ের ব্যাখ্যা আছে।

AI-সাহায্য প্রস্পট এবং মান-নিয়ন্ত্রণ

AI দিয়ে প্রশ্ন তৈরি করার জন্য একটি স্পষ্ট প্রস্পট থাকবে। উদাহরণস্বরূপ: “Generate 30 MCQs (10 Easy, 10 Medium, 10 Hard) on [Section X topics] in Python 3. Include question, code snippet (if any), 4 options (exactly one correct), difficulty label, topic label, and a detailed explanation.” AI প্রথমে প্রশ্নগুলি ড্রাফট করবে। তবে প্রশ্ন সরাসরি প্রকাশ করা যাবে না। শিক্ষককে পর্যালোচনার সুযোগ দেওয়া হবে [1†L122-L124]। AI-জেনারেটেড প্রশ্নগুলির পাশপাশি নিম্নোক্ত বিষয়গুলোর দিকে নজর দেওয়া হবে:

- **Difficulty Calibration:** Easy প্রশ্নগুলো সাধারণ কনসেপ্ট তুলে ধরবে, Hard প্রশ্নগুলো বেশি চিন্তাশীল হবে। যদি প্রয়োজন হয়, তাহলে “Make Harder” বা “Make Easier” অপশন ব্যবহার করে প্রশ্ন জটিলতা সামঞ্জস্য করা যাবে।
- **One Correct Answer:** প্রতিটি MCQ-তে সঠিক উত্তরটি স্পষ্ট থাকতে হবে। AI থেকে যেকোনো দ্ব্যর্থতা এড়াতে সংশোধন করতে হবে [1†L122-L124]।
- **পাইথন ও সঙ্গতি:** কোড স্নিপেটগুলো Python 3 এ সঠিকভাবে রান করে। কোনো সিম্যাটিক বা সিনটাক্সের ত্রুটি থাকলে সংশোধন করতে হবে।
- **Distractors (ভুল উত্তর):** ভুল অপশনগুলো বাস্তবসম্মত ও বিভ্রান্তিকর হওয়া উচিত; একটুও অসম্মানজনক নয়।
- **ব্যাখ্যামূলক উত্তর:** প্রতিটি প্রশ্নের শেষে ব্যাখ্যা বুঝিয়ে দিতে হবে কেন সঠিক উত্তরটি সঠিক এবং ভুল অপশনগুলো ভুল।
- **টপিক ও আইডি:** `topic` ফিল্ড হিসেবে সংশ্লিষ্ট বিষয় এবং ইউনিক `id` ঠিক করা থাকবে। ডুপ্লিকেট প্রশ্ন বা খুব মিল থাকা প্রশ্ন এড়াতে হবে।

এই সব মাপকাঠি নিশ্চিত করে শিক্ষক একটি মান-নিয়ন্ত্রণ চেকলিস্ট অনুসরণ করবেন।

প্রশ্নের মান নিয়ন্ত্রণ চেকলিস্ট

- একক সঠিক উত্তর: প্রতিটি MCQ-তে যেন নির্দিষ্টভাবে একটিই সঠিক থাকে। (কোনও দ্ব্যর্থতা নেই)।
- **Python 3 সঙ্গতি:** সব কোড স্নিপেট Python 3-এ সঠিকভাবে চলে। প্রশ্নে ইচ্ছাকৃত সিনটাক্স এরর ছাড়া অন্য ত্রুটি থাকা চলবে না।
- **দুর্নীতিমূলক প্রশ্ন:** অস্পষ্ট বা বিভ্রান্তিকর শব্দ ব্যবহার নেই। প্রশ্ন ও অপশনগুলি স্পষ্টভাবে লেখা।
- **কমপ্লেক্সিটি অনুযায়ী ডিফিকাল্টি:** ডিফিকাল্টি লেভেল প্রশ্নের কঠিনতার সাথে মানানসই। (Easy সহজ কনসেপ্ট, Hard বিশ্লেষণাত্মক)।
- **ব্যালেন্সড ডিস্ট্রাক্টর:** ভুল অপশনগুলো ভাল distractor, অর্থাৎ উদ্দেশ্যমূলকভাবে বিভ্রান্ত করতে পারে কিন্তু ভুল। কোনো অপশন যেন স্বতঃসিদ্ধ না হয়।
- **সঠিক ব্যাখ্যা:** প্রত্যেক প্রশ্নে কেন উত্তরটি সঠিক এবং ভুল অপশনগুলো কেন ভুল তার ব্যাখ্যা দিতে হবে। শুধুমাত্র “ভুল” লেখা নয়, ধারণা বোঝাতে হবে।
- **টপিক মিল:** প্রশ্নটি কুইজের সিলেবাস/টপিকের সাথে সরাসরি সম্পর্কিত।

- ইউনিক আইডি: প্রতিটি প্রশ্নের জন্য নির্দিষ্ট `id` ব্যবহার।
- ভুল/অপ্রাসঙ্গিক কোড না থাকা: কোনো কোড-অংশ যেন প্রশ্নের সাথে সংশ্লিষ্ট নয় না হয়। কোড ইনপুট-আউটপুট যাচাই করা হয়েছে।
- কনটেন্ট বৈচিত্র্য: শিক্ষার্থীদের বিভিন্ন ধরনের প্রশ্ন (তত্ত্বগত, কোড-আউটপুট, ট্রেসিং) দেওয়া হয়েছে এবং একই কনসেপ্টের পরিবর্তিত উপস্থাপনা আছে।

এই চেকলিস্ট অনুসরণে শিক্ষক AI-উৎপন্ন প্রশ্নগুলো যাচাই ও সম্পাদনা করবেন **【1†L122-L124】**। শুধুমাত্র যাচাই হয়ে যাওয়ার পরেই “Publish Quiz” পাবে।

স্কোরিং এবং টাইমার নীতি

প্রতি প্রশ্নের জন্য ২০ সেকেন্ড টাইমার থাকবে (ডাউন কাউন্ট)। ভিজিবিলাটি এবং ফোকাসের জন্য স্পষ্ট ডিজাইন রাখা উচিত: যেমন বড় সংখ্যায় টাইমার, প্রপ্রেস বার বা রঙ, এবং প্রয়োজনে ওয়াইবার অনাউন্সমেন্ট। iSpring এর মত উৎসের মতে, টাইমার থাকা প্রতিযোগিতা এড়াতে সাহায্য করে **【3†L50-L53】**। প্রতিটি প্রশ্ন শুরুতেই টাইমার ২০ থেকে শুরু হবে এবং ০-তে পৌঁছালে স্বয়ংক্রিয়ভাবে অপারিকলিত উত্তর হিসেবে চিহ্নিত হবে। উত্তর পড়ার ও উত্তর দেয়ার সময়ে ক্রটি এড়াতে সাবধানে ইভেন্ট হ্যান্ডল করতে হবে (যেমন, একই সময়ে উত্তর নির্বাচন ও টাইমআউট হলে দ্বিগুণ ইভেন্ট এড়ানো)। React-এ, স্টেটটাইমআউট বা Web Worker দিয়ে যথাযথ কাউন্টডাউন রাখতে হবে, কারণ ব্যাকগ্রাউন্ড ট্যাবে `setInterval` থ্রোটল হয়ে যেতে পারে **【12†L49-L57】** **【12†L61-L64】**।

মার্কিং: সঠিক উত্তরে +১। ভুল উত্তর দিতে কোনো দণ্ড থাকবে না যদি না ভুলের পরিমাণ কুইজের ৩০%-এর বেশি হয়। অর্থাৎ ৩০ প্রশ্নে ৯ টির বেশি ভুল করলে প্রতিটি ভুলের জন্য -0.25 যোগ হবে। (অস্বীকার করা যায় যে, ১০ বা তার বেশি ভুল হলে নেগেটিভ মার্কিং শুরু)। নির্ধারিত নিয়মগুলো কুইজ শুরুতে এবং ফলাফলের সাথে প্রদর্শন করতে হবে। ফলাফলের বক্সে সঠিক, ভুল, নিশ্চয়, অ্যাকুরেসি ইত্যাদি স্পষ্ট দেখানো হবে।

ফলাফল ও বিশ্লেষণ

কুইজ শেষে **Final Score** পেজ দেখানো হবে। এতে থাকবে: **Final Score** (যেমন ২৩.৭৫/৩০) এবং নিচে বিশদ আইটেম: মোট প্রশ্ন, সঠিক সংখ্যা, ভুল সংখ্যা, নিশ্চয় সংখ্যা, অ্যাকুরেসি (%)। উদাহরণস্বরূপ:

Score Summary	
Total Questions	30
Correct	25
Wrong (no penalty)	8
Unanswered (Timed out)	1
Accuracy	83.3%

সাথে **Difficulty Breakdown: Easy/Medium/Hard** প্রতি বিভাগে কতটি সঠিক। যেমন:

- Easy: 9/10, Medium: 8/10, Hard: 6/10।

এছাড়া **Topic-wise Performance** বা **Weak Areas** বোঝাতে একটি সারণি বা বার-গ্রাফ থাকতে পারে (যেখানে Topics列 এবং সঠিকতার শতাংশ দেখাবে)।

চূড়ান্ত স্কোরিং গণনা UI নয়, বরং জমা করা উত্তরগুলোর স্টেট থেকে হিসাব হবে। আমরা যেমন KISSmetrics এ দেখেছি, শুধুমাত্র গড় স্কোর নয় বরং স্কোরের বিতরণও গুরুত্বপূর্ণ **【20†L219-L224】**। সেই কারণে তুলনামূলক বিশ্লেষণের জন্য গড় ছাড়াও বিভাগীয় ভাগ (অংশগ্রহণকারী শিক্ষার্থীদের মধ্যে কতজন পাস/ফেল করেছে) রাখা যেতে পারে।

প্রশ্ন-ভিত্তিক রিভিউ: রেজাল্টের পর, প্রতিটি প্রশ্নের জন্য একটি রিভিউ পেজ থাকবে। সেখানে থাকবে প্রশ্ন নম্বর, প্রশ্ন (ও কোড), সব অপশন, ব্যবহারকারীর উত্তর (ফলাফলের সাথে ✓/✗), সঠিক উত্তর চিহ্নিত, এবং বিস্তারিত ব্যাখ্যা। টপিক ও ডিফিকাল্টি স্তরও দেখানো হবে। উদাঃ:

প্রশ্ন 12:

Your Answer: B (Incorrect)

Correct Answer: D

তুমি ভুলেছ: কারণ একটি স্টেপ মিস করেছিলেন...

বিষয়: List Comprehension | ডিফিকাল্টি: Medium

এইভাবে শিক্ষক ও শিক্ষার্থী উভয়ই বুঝতে পারবে কোন অংশে দুর্বলতা রয়েছে এবং কেন ভুল হয়েছে।

পরীক্ষণ ও এজ-কেস

- টাইমার রেস কন্ডিশন: নিশ্চিত করতে হবে যে ২০ সেকেন্ডের পরে প্রশ্ন আর উত্তর দেওয়া যায় না। একই সময়ে ক্লিক এবং টাইমআউট ইভেন্ট একাধিকবার ফায়ার হলে সঠিকভাবে হ্যান্ডল করুন (উদাঃ একটি `ISR` বা ডিবাউন্স ইভেন্ট)।
- তারাতারি ক্লিক প্রতিরোধ: কোনো অপশনে বহুবার ক্লিক করলে একবারই উত্তর রেজিস্টার হবে। ক্লিক করার সাথে সিলেক্টেড স্টেট ব্যাখ্যা করুন।
- ট্যাব সুইচ: `Page Visibility API` এর মাধ্যমে নোট করুন যে ব্যবহারকারী ট্যাব পরিবর্তন করেছে `【19†L184-L193】`। এটি স্বয়ংক্রিয় ফলাফল নয়, তবে ডেটা লগ করে রাখুন। (যেমন, `“tab_hidden”` ইভেন্ট লগ করা)।
- ফুলস্ক্রিন বাদ: কুইজ চলাকালীন `fullscreen mode` বাধ্যত করুন, যাতে শিক্ষার্থী অন্য ট্যাবে কম যেতে পারে।
- পেজ রিফ্রেশ: রিফ্রেশ করলে কুইজ শুরু থেকে রিসেট হতে পারে; ব্যবহারকারীকে সতর্ক বার্তা দেখানো বা অ্যাপের হালাত-আহা দেখে বন্ধ করা যেতে পারে।
`Phase 1` এ সহজতর করতে `“Refresh = exit”` ধরণ থাকতে পারে।
- ডেটা অঞ্চলতা: প্রশ্ন ডেটায় ফাঁকা বা ভুল ফরম্যাট চেক করুন। ইউনিক `id` ও অপশন ৪টি নিশ্চিত করুন।
- নেগেটিভ মার্কিং প্রেশহোল্ড: ভুল উত্তর >9 টি হলে -0.25 স্কোর যোগ হচ্ছে কিনা যাচাই করুন।
- ক্যালকুলেশন ঠিকঠাক: চূড়ান্ত স্কোর `UI` থেকে নয়, স্টোরড স্টেট (উত্তর লিস্ট) থেকে গণনা হবে। দ্ব্যর্থতা রোধে পরীক্ষার শেষে পুনরায় লজিক রান করে ফলাফল দেখান।
- পুনঃরায় উপপাদন: একই প্রশ্ন সেটের পুনরাবৃত্তি এড়াতে ভিন্ন-ক্রমে প্রশ্ন ও অপশন ওর্ডার করে দেখাতে হবে, কিন্তু ভিন্ন অ্যাটম্পটে `Q/A` মিস্স রাখতে হবে।
- ব্যাকগ্রাউন্ডে ইন্টারলক: টাইমার `Web Worker` ব্যবহার না করলে `inactive tab` এ `setInterval` ধীর হতে পারে `【12†L49-L57】`।
পরিপূর্ণ অভিজ্ঞতার জন্য `Workers` ব্যবহার বিবেচনা করা যেতে পারে।

এইসব পরীক্ষা ও চেকলিস্ট পূরণ করলে ফ্রন্টএন্ড ভুল কম দেখাবে ও প্রাটফর্মের নির্ভরযোগ্যতা বাড়বে।

MVP রোডম্যাপ ও সময়রেখা

ফেজ ১-এর এমভিপি ডেলিভারি জন্য আনুমানিক সময়রেখা (একজন ডেভেলপার ভিত্তিতে) নিম্নরূপ:

পর্যায় (Milestone)	কাজের বিবরণ	আনুমানিক সময় (দিন)
প্রজেক্ট সেটআপ	<code>Next.js + TypeScript + Tailwind</code> কনফিগারেশন, রিপোজিটরি ইনিশিয়ালাইজ	2
শিক্ষক ইন্টারফেস	ড্যাশবোর্ড, সেকশন/সেট কার্ড, <code>Quiz Creation UI</code> , <code>AI</code> রিভিউ ইন্টারফেস	4
ছাত্র ইন্টারফেস	কোড-জয়েন পেজ, ওয়েটিং রুম, <code>Quiz</code> পর্দা (প্রশ্ন + অপশন)	3
কুইজ ইঞ্জিন	প্রশ্ন ড্রপডাউন, <code>20s</code> টাইমার, স্বয়ংক্রিয় পরবর্তী বিষয়, স্টেট ম্যানেজমেন্ট	4
স্কোরিং ও ফলাফল	মার্কিং ক্যালকুলেশন (নেগেটিভ মার্কিং সহ), রেজাল্ট পেজ ও <code>Analytics UI</code>	3
অ্যাক্সেসিবিলিটি ও টেস্টিং	<code>WCAG</code> ফিচার (<code>ARIA</code> , কীবোর্ড সহায়ক), রেসপন্সিভ, লোকাল টেস্টিং ও বাগ ফিক্সিং	2
বাফার/রিফাইনমেন্ট	বাফার টাইম: ছোটখাটো পরিবর্তন, ডকুমেন্টেশন, টিউটোরিয়াল	2
মোট		20 দিন

প্রথম মাইলস্টোনে প্রকল্পের ভিত্তি গড়ে ওঠবে (দিন 1-2)। পরবর্তী সপ্তাহে শিক্ষক ও ছাত্র উভয়ের জন্য UI তৈরি এবং কুইজ লজিক (দিন 3-10)। তৃতীয় সপ্তাহে স্কোরিং ও ফলাফল এবং QA/টস্টিং (দিন 11-18)। চতুর্থ সপ্তাহের শুরুতে অবশিষ্ট কাজ সেরে ফাইনাল ফর্মালিটি (দিন 19-20)। মোটামুটি ৩-৪ সপ্তাহে সম্পূর্ণ করা সম্ভব মনে হচ্ছে।

নিরাপত্তা ও সততা বিবেচনা

- অনলাইন পরীক্ষার সততা: শিক্ষার্থীদের ফোকাস ট্র্যাক করতে **visibilitychange** ও **blur** ইভেন্ট ব্যবহার করা যেতে পারে **【19†L184-L193】**। যেমন, ট্যাব-বাইরের সময় লগ রাখা। তবে সতর্ক থাকতে হবে যে, প্রত্যেক ফোকাস-লস 'চুরি' নয় **【19†L219-L224】**। উদ্দেশ্য এড়াতে সাধারণত আরো এন্টিভেশন/ইনফরমেশন দেয়া হয়, পুরোপুরি নির্মূল নয়।
- লকডাউন মোড (**Phase 1**): পরীক্ষার সময় পূর্ণাঙ্গিন বাধ্যতামূলক। কপি-পেস্ট/প্রিন্ট নিষিদ্ধ করা। এগুলি সম্পূর্ণ প্রতিরোধ নয়, তবে বাধা দেওয়া যায়। ভবিষ্যতে Phase ২-এ আরও শক্তিশালী প্রোটেক্টিং বা ব্রাউজার লকডাউনের (যেমন Respondus) ধরণ যোগ করা যেতে পারে।
- ডেটা সিকিউরিটি (**Phase 1**): কোন ব্যাকএন্ড না থাকলেও জাভাস্ক্রিপ্টে সংবেদনশীল তথ্য রাখার আগে এনক্রিপশন বিবেচনা করতে হবে (যেমন, প্রশ্নের ব্যাখ্যা গোপন রাখা বা সঠিক উত্তরের ইনডেক্স ক্রায়েন্টে রাখার ভয়)। Phase 2-এ ছাত্র তথ্য এবং ফলাফলের জন্য অথেন্টিকেশন ও ব্যাকএন্ড নিরাপদভাবে আনা জরুরি হবে।
- সেশন নিরাপত্তা: প্রত্যেক কুইজ কোড একটি-একটি করে থাকবে (অনন্য) এবং একই কোডে একাধিক ক্রায়েন্ট না যুক্ত করার ব্যবস্থা রাখুন।
- HTTPS** ব্যবহৃত: সব যোগাযোগ এনক্রিপ্টেড রাখা। (Phase 1 এ ফ্রন্টএন্ড হলেও হোস্টিং HTTPS থাকার কথা)।
- নকল প্রতিরোধ: প্রশ্নব্যাংকে বড় বিশাল উত্তরব্যাংক থাকলে প্রতিটি পরীক্ষা আলাদা প্রশ্ন তুলে দেওয়া যায়, যাতে যেকোনো এক সেটের Memorize-এর সুযোগ নেই। (Phase 1-এ ১২ সেট হলেও পরবর্তী Phase-এ বড় ব্যাংক হতে পারে)।
- পূর্ণ সততা (**Phase 2**): ভবিষ্যতে BIOMETRIC অথবা লাইভ প্রোটেক্টিং সফটওয়্যার ইন্টিগ্রেশন করে সনাক্তকরণ বাড়ানো যেতে পারে।

এগুলিতে [19] তে মত, ট্যাব-সুইচ ও ভিজিবিলাটি চেক করা যেতে পারে, কিন্তু ভুল ফলাফলে ফেল করার আগে মানবজ্ঞান ব্যবহার করতে হবে **【19†L219-L224】**। নিরাপত্তা-বিষয়ক আপগ্রেডের দিকে লক্ষ্য রেখে অবকাঠামো তৈরি করা উচিত।

মূল্যায়ন সূচক (Evaluation Metrics)

শিক্ষককে প্র্যাটফর্মের কার্যকারিতা বোঝাতে নিম্নলিখিত মেট্রিক্স নজর দেয়া যেতে পারে:

- গড় স্কোর ও মার্ক বিতরণ: প্রশ্নের গড় স্কোর হিসাব (মোট স্কোরের গড়) এবং স্কোরের বিতরণ। যেমন পাশের ভাগ এবং ফেলের ভাগ। **KISSmetrics** মতে, মূল্যায়নে শুধু গড় নয় স্কোরের ডিস্ট্রিবিউশনও গুরুত্বপূর্ণ **【20†L219-L224】**।
- সঠিকতা (%) – মোট সঠিক উত্তরের শতাংশ (Accuracy)। উদাহরণস্বরূপ, ৩০ টি প্রশ্নে সঠিক ২৫ হলে অ্যাকুরেসি ৮৩.৩%।
- প্রশ্ন-গতি (Time per Question): গড় উত্তর প্রদানের সময়। এ থেকে বোঝা যায় কোন প্রশ্নে বেশি সময় লাগছে (সম্ভবত বেশি কঠিন)। উচ্চ সময়ের প্রশ্নগুলো চিহ্নিত করা যেতে পারে।
- অংশগ্রহণ হার (Engagement): কতজন শিক্ষার্থী পরীক্ষায় বসছে। উপস্থিত থাকা শিক্ষার্থীর শতাংশ (যেকজন জয়েন করেছে তাদের মধ্যে পরীক্ষায় অংশগ্রহণকারী কতজন)।
- বিভাগ ভিত্তিক পারফরম্যান্স: সহজ/মাঝারি/কঠিন প্রতিটি বিভাগে শতকরা সঠিকতা। এই হিসেবে কোন ধাপের প্রশ্নগুলিতে ছাত্ররা ভাল বা খারাপ করছে।
- টপিক ভিত্তিক দুর্বলতা: প্রতিটি টপিকে (যেমন List, Tuple ইত্যাদি) সঠিকতার শতাংশ। কম পারফরম্যান্সের টপিকগুলো স্বয়ংক্রিয়ভাবে “Weak Areas” হিসেবে তুলে ধরা যেতে পারে। যেমন: “Nested Comprehension” বা “Dictionary Methods” এ কম সঠিকতা থাকলে শিক্ষককে সতর্ক করা।
- টাইম টু আসেসমেন্ট (Reaction Time): শিক্ষার্থী কিভাবে কুইজের প্রতিটি অংশে সময় ব্যবহার করছে – অ্যান্টিভ লার্নিং টাইম (মনিটর করতে পারে, যেমন মাউস মুভমেন্ট বা ক্লিকিং) **【20†L219-L224】** **【20†L190-L198】**। (এতে বোঝা যায় প্রকৃত অংশগ্রহণের পরিমাণ)

এসব মেট্রিক্সে ফ্রন্টএন্ড থেকে জেনারেট করা ডাটা এবং ছাত্রদের উত্তর দিয়ার সময়ে লগড ডাটা ব্যবহৃত হবে। যেমন, Tab-আউট/ফুলজিন exভেন্ট বিশ্লেষণ করে শিক্ষকের জন্য integrity রিপোর্ট তৈরি করতে পারে। এছাড়া ছাত্রদের ক্রমাগত উন্নতি লক্ষ করার জন্য প্রতিটি অংশের ফলাফল সময় ধরে খতিয়ে দেখা যেতে পারে (যেমন গত ৩টি কুইজে কোন বিষয়ে প্রবলেন হচ্ছে)।

উল্লেখ্য: এই ফেজে প্রধান লক্ষ্য হলো শিক্ষককে একটি পরিপূর্ণ কুইজ মডিউল দেওয়া যা তাদের জন্য বাকি পরীক্ষার কাজ অর্ধেক কমিয়ে দেয়। উন্নয়নের পরের ধাপে (Phase 2) এ আরো ইউজার অথেন্টিকেশন, বড় প্রশ্ন ব্যাংক, এবং উন্নত অ্যান্টি-চিট ব্যবস্থা যুক্ত করা হবে। Phase 1 এর MVP তে শিক্ষক ও শিক্ষার্থী উভয়ের জন্য স্পষ্ট ও ব্যবহারযোগ্য একটি সিস্টেম পাকা করা হবে, যাতে পরবর্তী ধাপে বাড়তি ফিচার বসানো সহজ হয়।